

docker

Test de repaso — parte 2: imágenes y Dockerfile

100 preguntas · 4 opciones, 1 correcta

Cómo usar este banco de preguntas

- Cada diapositiva contiene una pregunta con 4 opciones (A-D), solo 1 correcta.
- No incluye las respuestas: úsalo para practicar de cara al cuestionario. TODAS las preguntas del cuestionario saldrán de este banco de preguntas.

Bloque 1

Dockerfile y FROM

Pregunta 1

¿Cuál es la función principal de un Dockerfile?

- A) Describir, en pasos, cómo construir una imagen Docker
- B) Definir redes virtuales entre contenedores
- C) Configurar el firewall del host
- D) Orquestar varios contenedores a la vez

Pregunta 2

Un Dockerfile empieza directamente con `WORKDIR /app` sin ninguna instrucción `FROM` antes. ¿Qué ocurre al intentar construir la imagen?

- A) Se construye igual, usando `ubuntu:latest` como base por defecto
- B) Docker da un error: toda imagen (salvo `scratch`) necesita una instrucción `FROM` antes de otras instrucciones
- C) Funciona pero el contenedor no tiene sistema de ficheros
- D) Docker pregunta interactivamente qué imagen base usar

Pregunta 3

¿Qué significa `FROM scratch` en un Dockerfile?

- A) Que se usa una variante ligera de Ubuntu
- B) Que se descarta el contenido anterior del Dockerfile
- C) Que se parte de una imagen base completamente vacía, sin sistema operativo alguno
- D) Que la imagen se construye sin conexión a internet

Pregunta 4

Si un Dockerfile usa `FROM node:20-alpine`, pero más adelante necesitas herramientas de compilación que no vienen en Alpine, ¿cuál es el problema más probable?

- A) Ninguno, Alpine incluye todas las herramientas de cualquier distribución
- B) Las imágenes Alpine no permiten instalar paquetes nunca
- C) Alpine solo funciona con lenguajes interpretados, nunca compilados
- D) Alpine usa `musl` y un conjunto de paquetes mínimo; puede faltar alguna herramienta que sí viene en imágenes basadas en Debian/Ubuntu

Pregunta 5

¿Qué ventaja tiene usar una imagen base oficial de un lenguaje (p. ej. `python:3.12-slim`) en vez de partir de `ubuntu` e instalar el lenguaje a mano?

- A) La imagen oficial ya trae el runtime instalado y configurado, mantenido por la comunidad/el proyecto, ahorrando pasos y errores
- B) Ninguna, da exactamente igual
- C) Las imágenes de `ubuntu` no pueden ejecutar Python
- D) Las imágenes oficiales de lenguaje no se pueden usar en producción

Pregunta 6

Dos compañeros construyen la misma imagen semanas distintas a partir de `FROM python:3`. Uno obtiene Python 3.11 y el otro Python 3.12. ¿Por qué?

- A) Es un fallo de Docker
- B) `python:3` es un tag poco específico que puede apuntar a una versión menor distinta con el tiempo; conviene fijar más (p. ej. `python:3.11-slim`)
- C) Depende del sistema operativo del host, no del tag
- D) No es posible, siempre da la misma versión

Pregunta 7

¿Qué parte de `FROM node:20-alpine AS builder` identifica el tag?

- A) `node`
- B) `builder`
- C) `20-alpine`
- D) `AS`

Pregunta 8

Un Dockerfile tiene `FROM ubuntu` (sin tag). ¿Qué versión de Ubuntu se usará?

- A) La LTS más reciente siempre, fijada por Docker
- B) Ubuntu 18.04 siempre, por compatibilidad
- C) Da error porque Ubuntu exige tag obligatorio
- D) La que la imagen `ubuntu` tenga marcada como `latest` en ese momento en Docker Hub, que puede cambiar

Bloque 2

COPY, ADD y .dockerignore

Pregunta 9

¿Qué diferencia clave hay entre `COPY` y `ADD` que justifica preferir `COPY` por defecto?

- A) `ADD` tiene comportamientos "mágicos" adicionales (extraer tar, descargar URLs) que pueden sorprender si no se esperan
- B) `COPY` es más lenta
- C) `ADD` no puede copiar carpetas
- D) `COPY` no existe en versiones recientes de Docker

Pregunta 10

Tienes `COPY *.json ./` en un Dockerfile. Si el contexto de build no contiene ningún fichero `.json`, ¿qué ocurre?

- A) Se copia un fichero vacío llamado `*.json`
- B) Docker da un error porque el patrón no encuentra coincidencias
- C) Se omite silenciosamente y el build continúa sin copiar nada
- D) Docker lo sustituye automáticamente por `package.json`

Pregunta 11

¿Por qué conviene tener un `.dockerignore` incluso si ya existe un `.gitignore` con contenido parecido?

- A) Porque Docker lee automáticamente el `.gitignore`, así que es redundante
- B) Porque `.dockerignore` sustituye a `.gitignore`
- C) Porque son ficheros independientes: `.gitignore` afecta a git, `.dockerignore` afecta al contexto de build; conviene mantener ambos
- D) No conviene, son incompatibles entre sí

Pregunta 12

Un `.dockerignore` contiene la línea `*.log`. Si copias con `COPY . .` una carpeta y en una capa anterior de la imagen ya existía `app.log` (de un build previo), ¿qué sucede en el nuevo build?

- A) `.dockerignore` elimina `app.log` de la imagen aunque ya estuviera en una capa anterior
- B) El build falla
- C) Se ignora `.dockerignore` si ya existe una capa previa
- D) `.dockerignore` solo afecta a qué se envía como contexto de build en ese `COPY`; no puede borrar lo que ya quedó en capas anteriores de la imagen

Pregunta 13

¿Qué pasa si pones `COPY . .` como la primera instrucción después de `WORKDIR` , antes de `RUN npm install` ?

- A) Cualquier cambio en el código (aunque no cambien las dependencias) invalida la caché de esa capa y obliga a reinstalar dependencias en cada build
- B) Nada relevante, da igual el orden
- C) `npm install` se ejecuta automáticamente antes de `COPY`
- D) El build es más rápido así

Pregunta 14

¿ `ADD` puede usarse para descomprimir automáticamente un fichero `.tar.gz` local en el destino de la imagen?

- A) No, nunca
- B) Sí, es una de las funciones adicionales de `ADD` frente a `COPY`
- C) Solo con `COPY`, nunca con `ADD`
- D) Solo si el fichero está en una URL remota

Pregunta 15

Un alumno escribe `COPY https://miweb.com/fichero.txt /app/` pensando que descargará el fichero. ¿Qué es correcto?

- A) `COPY` funciona igual que `ADD` con URLs
- B) Docker lo descarga pero lo guarda con otro nombre
- C) `COPY` no admite URLs como origen; para eso haría falta `ADD` o un `RUN curl / wget`
- D) Esa sintaxis no es válida en ningún caso, ni con `ADD`

Pregunta 16

¿Cuál es el riesgo principal de hacer `COPY . .` en el contexto raíz de un proyecto sin `.dockerignore`, si ahí también viven las carpetas `.git` y `node_modules`?

- A) Ninguno, Docker los excluye siempre automáticamente
- B) El build fallaría directamente
- C) `node_modules` nunca puede copiarse con `COPY`
- D) Se copiarían a la imagen carpetas pesadas e innecesarias, aumentando tiempo de build y tamaño final

Bloque 3

RUN, CMD y ENTRYPOINT

Pregunta 17

¿En qué momento se ejecuta una instrucción `RUN` ?

- A) Una vez, durante la construcción (`docker build`) de la imagen
- B) Al arrancar cada contenedor creado desde la imagen
- C) Solo cuando se hace `docker exec`
- D) Nunca, es solo documentación

Pregunta 18

Un Dockerfile define `ENTRYPOINT ["python3", "app.py"]` y `CMD ["--debug"]`. Al ejecutar `docker run mi-imagen`, ¿qué comando se ejecuta realmente?

- A) `python3 app.py`
- B) `python3 app.py --debug`
- C) Solo `--debug`
- D) Da error porque no se pueden combinar

Pregunta 19

Con el Dockerfile anterior (`ENTRYPOINT ["python3","app.py"]` , `CMD ["--debug"]`), si ejecutas `docker run mi-imagen --verbose` , ¿qué ocurre?

- A) Se ejecuta `python3 app.py --debug --verbose`
- B) Se ignora el argumento `--verbose`
- C) Se ejecuta `python3 app.py --verbose` (el argumento de `docker run` sustituye al `CMD` , pero no al `ENTRYPOINT`)
- D) Se sustituye el `ENTRYPOINT` por `--verbose`

Pregunta 20

Si un Dockerfile solo tiene `CMD ["python3", "app.py"]` (sin `ENTRYPOINT`), y ejecutas `docker run mi-imagen bash`, ¿qué pasa?

- A) Se ejecutan ambos comandos, `python3 app.py` y luego `bash`
- B) Da error, no se puede sobrescribir un `CMD`
- C) `bash` se añade como argumento a `python3 app.py`
- D) El comando pasado en `docker run ( bash )` sustituye completamente al `CMD` de la imagen

Pregunta 21

¿Por qué se recomienda la "forma exec" (`CMD ["nginx", "-g", "daemon off;"]`) frente a la "forma shell" (`CMD nginx -g "daemon off;"`)?

- A) La forma exec no pasa por una shell intermedia, por lo que las señales (como `SIGTERM` al parar el contenedor) llegan directamente al proceso principal
- B) Son exactamente equivalentes en todos los casos, da igual cuál usar
- C) La forma shell es más rápida de escribir siempre
- D) La forma exec no admite argumentos

Pregunta 22

Si un Dockerfile tiene dos instrucciones `CMD` distintas, una tras otra, ¿qué ocurre al arrancar el contenedor?

- A) Se ejecutan ambas, la primera y luego la segunda
- B) Solo tiene efecto la última `CMD` ; las anteriores quedan completamente ignoradas
- C) Da un error de sintaxis al hacer `docker build`
- D) Se ejecuta la primera y la segunda se usa como argumentos de la primera

Pregunta 23

¿Qué usarías en el Dockerfile si quieres que el contenedor siempre ejecute un script fijo de arranque, sin que un usuario pueda sustituirlo fácilmente con `docker run imagen otro-comando` ?

- A) `CMD` en forma shell
- B) `RUN`
- C) `ENTRYPOINT`
- D) `WORKDIR`

Pregunta 24

Un script de arranque necesita hacer tareas antes de lanzar el proceso principal (p. ej. esperar a una base de datos) y luego ejecutar lo que el usuario pase en `docker run`. ¿Qué patrón encaja mejor?

- A) Usar `RUN` para esas tareas de espera
- B) No es posible hacer eso en Docker
- C) Poner todo en `WORKDIR`
- D) Usar un script como `ENTRYPOINT` que haga esas tareas y al final ejecute `exec "$@"` con los argumentos recibidos

Pregunta 25

Si ejecutas `docker run mi-imagen echo "hola"` y la imagen tiene `ENTRYPOINT ["python3", "app.py"]` (sin `CMD`), ¿qué sucede?

- A) Se ejecuta `python3 app.py echo hola`, pasando `echo` y `hola` como argumentos a `app.py`
- B) Se ejecuta `python3 app.py` y se ignoran `echo "hola"`
- C) Se ejecuta solo `echo "hola"`
- D) Da error porque no se puede combinar `ENTRYPOINT` con argumentos extra

Pregunta 26

¿Qué diferencia hay entre `RUN` y `CMD / ENTRYPOINT` en cuanto a cuándo tienen efecto?

- A) `RUN` dura para siempre ejecutándose en segundo plano
- B) `RUN` deja su efecto grabado en una capa de la imagen durante el build y no se repite al arrancar contenedores; `CMD / ENTRYPOINT` se ejecutan cada vez que arranca un contenedor
- C) Son exactamente lo mismo, solo cambia el nombre
- D) `CMD` solo se ejecuta la primera vez que se arranca el contenedor

Bloque 4

WORKDIR, ENV, EXPOSE y USER

Pregunta 27

Si un Dockerfile tiene `WORKDIR /app` y luego `WORKDIR config`, ¿cuál es el directorio de trabajo final?

- A) `/config`
- B) Solo `config`, sin relación con `/app`
- C) `/app/config`
- D) Da error, no se puede usar `WORKDIR` dos veces

Pregunta 28

¿Por qué `WORKDIR /app` es preferible a `RUN mkdir /app && RUN cd /app` para las instrucciones siguientes?

- A) Son equivalentes exactamente
- B) `RUN cd` no está permitido en Docker
- C) `WORKDIR` es más lento
- D) `cd` dentro de un `RUN` solo afecta a esa instrucción concreta; `WORKDIR` persiste el directorio para todas las instrucciones posteriores (incluido `CMD`)

Pregunta 29

Un Dockerfile define `ENV NODE_ENV=production`. Al ejecutar `docker run -e NODE_ENV=development mi-imagen`, ¿qué valor tendrá la variable dentro del contenedor?

- A) `development`, porque `-e` en `docker run` sobrescribe el valor definido con `ENV` en la imagen
- B) `production`, porque `ENV` del Dockerfile tiene prioridad siempre
- C) Ambos valores a la vez, concatenados
- D) Ninguno, hay un conflicto y la variable queda vacía

Pregunta 30

¿Qué diferencia hay entre declarar una variable con `ENV` en el Dockerfile y usar un argumento `ARG` ?

- A) Son exactamente lo mismo
- B) `ENV` persiste en la imagen final y está disponible en los contenedores en ejecución; `ARG` solo existe durante el build y no queda en la imagen final por defecto
- C) `ARG` siempre tiene prioridad sobre `ENV`
- D) `ENV` solo puede usarse dentro de `RUN`

Pregunta 31

¿Qué hace exactamente `EXPOSE 443` en un Dockerfile?

- A) Publica automáticamente el puerto 443 del contenedor hacia el host, como si fuera `-p`
- B) Configura un certificado HTTPS automáticamente
- C) Documenta, a nivel de imagen, que el contenedor escucha en el puerto 443; no publica nada por sí sola
- D) Abre el puerto 443 en el firewall del host

Pregunta 32

Una imagen declara `EXPOSE 8080` pero al ejecutar `docker run mi-imagen` (sin `-p`), el navegador no puede acceder a `localhost:8080`. ¿Por qué?

- A) `EXPOSE` nunca funciona en ningún caso
- B) `EXPOSE` solo funciona con Nginx
- C) Hay que reiniciar Docker tras cada `EXPOSE`
- D) Falta mapear el puerto hacia el host con `-p 8080:8080` (o similar); `EXPOSE` por sí sola no publica nada

Pregunta 33

¿Para qué sirve la instrucción `USER` en un Dockerfile?

- A) Para indicar con qué usuario (distinto de `root`) se ejecutan las instrucciones siguientes y/o el contenedor en marcha
- B) Para definir el nombre de usuario de Docker Hub
- C) Para crear un usuario en el sistema operativo host
- D) Para definir el autor de la imagen en los metadatos

Pregunta 34

¿Por qué es recomendable usar `USER` para evitar ejecutar el proceso principal del contenedor como `root`, cuando no es estrictamente necesario?

- A) Porque `root` no existe dentro de los contenedores
- B) Por seguridad: limita el daño que podría hacer un proceso comprometido dentro del contenedor
- C) Porque `root` hace que el contenedor consuma más RAM
- D) No aporta ninguna ventaja real, es solo una moda

Bloque 5

docker build y contexto

Pregunta 35

¿Qué papel juega el `.` al final de `docker build -t mi-app:1.0 .` ?

- A) Indica la versión de Docker a usar
- B) Es el nombre del Dockerfile
- C) Indica el contexto de build: la carpeta (y su contenido) que se envía al daemon de Docker
- D) Indica que se debe ignorar la caché

Pregunta 36

¿Qué opción de `docker build` usarías si tu Dockerfile se llama `Dockerfile.prod` en vez de `Dockerfile` ?

- A) `-t Dockerfile.prod`
- B) `--name Dockerfile.prod`
- C) No es posible usar otro nombre
- D) `-f Dockerfile.prod`

Pregunta 37

El contexto de build de un proyecto incluye, por error, una carpeta de vídeos de varios GB no listada en `.dockerignore`. ¿Qué consecuencia directa tiene esto, aunque ningún `COPY` la referencie?

- A) Docker envía igualmente toda la carpeta al daemon como parte del contexto, haciendo el build innecesariamente lento
- B) Ninguna, solo se envía lo que se referencia en `COPY`
- C) El build falla inmediatamente
- D) Los vídeos se comprimen automáticamente

Pregunta 38

Durante un `docker build`, aparece el mensaje "Using cache" junto a la instrucción `RUN npm install`. ¿Qué implica?

- A) Que ha habido un error silencioso
- B) Que Docker ha reutilizado el resultado de un build anterior para esa capa, porque ni la instrucción ni las capas previas han cambiado
- C) Que se está descargando `npm install` de internet otra vez
- D) Que la imagen se va a reconstruir desde cero igualmente

Pregunta 39

¿Qué logra la opción `docker build --no-cache` ?

- A) Hacer el build más rápido reutilizando más capas
- B) Evitar que la imagen resultante se guarde en disco
- C) Forzar que todas las capas se reconstruyan desde cero, ignorando cualquier caché previa
- D) Construir la imagen sin necesidad de un Dockerfile

Pregunta 40

Ejecutas `docker build .` sin `-t`. ¿Puedes seguir usando la imagen resultante?

- A) No, sin `-t` no se genera ninguna imagen
- B) Se etiqueta automáticamente como `latest`
- C) Se borra automáticamente al terminar el build
- D) Sí, la imagen se crea igualmente pero solo queda identificada por su ID; convendría etiquetarla luego con `docker tag`

Pregunta 41

¿Qué comando usarías para ver, capa por capa, cuánto espacio aporta cada instrucción de un Dockerfile ya construido?

- A) `docker history`
- B) `docker images`
- C) `docker inspect --layers`
- D) `docker build --verbose`

Pregunta 42

Tras un `docker build -t app:1.0 .` que termina sin errores, ¿qué comando confirmaría que la imagen ya existe localmente con ese nombre y tag?

- A) `docker ps`
- B) `docker images`
- C) `docker network ls`
- D) `docker volume ls`

Bloque 6

Caché de capas y buenas prácticas

Pregunta 43

¿Por qué el orden de las instrucciones en un Dockerfile afecta directamente al tiempo de build en reconstrucciones sucesivas?

- A) No afecta, Docker siempre reconstruye igual
- B) Solo afecta a la velocidad de descarga de la imagen base
- C) Si una capa cambia, se invalida esa capa y todas las siguientes, obligando a rehacerlas aunque no hayan cambiado
- D) El orden solo importa en imágenes Alpine

Pregunta 44

¿Por qué es buena práctica copiar primero `package.json` e instalar dependencias, y copiar el resto del código después?

- A) Porque Docker lo exige obligatoriamente en ese orden
- B) Porque reduce el número de líneas del Dockerfile
- C) No aporta ninguna ventaja real, es solo estilo
- D) Porque así, si solo cambia el código (no las dependencias), la capa de `npm install` sigue en caché y no hace falta reinstalar nada

Pregunta 45

Tienes 6 instrucciones `RUN apt-get install paquete-X` seguidas, una por paquete. ¿Qué inconveniente tiene frente a agruparlas en una sola instrucción?

- A) Genera más capas de las necesarias y dificulta limpiar la caché de `apt` en el mismo paso, aumentando el tamaño final
- B) Ninguno, es exactamente igual de eficiente
- C) Hace que el build sea inseguro
- D) Impide usar `.dockerignore`

Pregunta 46

¿Qué técnica reduce el tamaño final de una imagen cuando se instalan herramientas de compilación que solo hacen falta durante el build, no en producción?

- A) Borrar manualmente ficheros dentro de un contenedor ya en ejecución
- B) Un build multi-stage: compilar en una etapa con esas herramientas y copiar solo el resultado final a la imagen de producción
- C) Usar siempre `FROM ubuntu:latest`
- D) No hay ninguna forma de reducirlo

Pregunta 47

¿Qué indica el principio de minimizar el número de capas en un Dockerfile (agrupando instrucciones relacionadas)?

- A) Que Docker limita a 10 capas como máximo
- B) Que el contenedor en ejecución consumirá menos RAM
- C) Que menos capas suelen traducirse en imágenes algo más pequeñas y builds más simples de razonar
- D) Que es obligatorio por las políticas de Docker Hub

Pregunta 48

¿Qué ventaja aporta fijar siempre una versión concreta en `FROM` (p. ej. `node:20.11-alpine`) en vez de `latest` o un tag muy genérico (`node:20`)?

- A) Ninguna diferencia práctica
- B) Las imágenes con versión concreta siempre pesan menos
- C) Es un requisito técnico obligatorio de Docker
- D) Builds reproducibles: la imagen resultante es la misma hoy y dentro de seis meses, evitando sorpresas por actualizaciones de la base

Pregunta 49

Un Dockerfile limpia la caché de `apt` (`rm -rf /var/lib/apt/lists/*`) en una instrucción `RUN` distinta de la que instala los paquetes. ¿Por qué esto no reduce el tamaño de la imagen como se esperaba?

- A) Cada `RUN` genera su propia capa; los ficheros ya quedaron escritos en la capa de instalación y borrarlos en una capa posterior no libera ese espacio en la imagen final
- B) Porque `rm` no funciona dentro de contenedores
- C) Porque `apt` no genera caché nunca
- D) Porque hace falta usar `ADD` en vez de `RUN`

Pregunta 50

Un equipo decide usar siempre imágenes base `-slim` o `-alpine` cuando es posible. ¿Qué contrapartida hay que tener en cuenta?

- A) Ninguna, son estrictamente mejores en todos los casos
- B) Pueden faltar herramientas/librerías del sistema que sí trae la imagen "completa", obligando a instalarlas aparte si se necesitan
- C) No se pueden usar en producción
- D) No permiten instalar paquetes con gestores de dependencias

Pregunta 51

¿Por qué se recomienda no dejar nunca una contraseña real "temporalmente" en una instrucción `RUN` o `ENV`, aunque luego se borre en una instrucción posterior?

- A) Porque `RUN` y `ENV` no admiten texto con símbolos
- B) Porque ralentiza el contenedor en ejecución
- C) Porque queda grabada en la capa correspondiente de la imagen y cualquiera que tenga la imagen puede inspeccionar su historial (p. ej. con `docker history`) y recuperarla
- D) Porque Docker la cifra automáticamente de todas formas, así que no hay riesgo real

Bloque 7

Multi-stage builds

Pregunta 52

¿Cuál es el objetivo principal de un build multi-stage?

- A) Construir varias imágenes totalmente independientes en paralelo
- B) Ejecutar varios contenedores a la vez desde un mismo Dockerfile
- C) Evitar tener que usar `docker build`
- D) Usar una etapa "pesada" para compilar/preparar, y copiar solo lo necesario a una etapa final más ligera

Pregunta 53

En `COPY --from=build /app/dist /usr/share/nginx/html`, ¿qué indica `--from=build`?

- A) Que se copia desde la etapa anterior del Dockerfile nombrada `build` (con `FROM ... AS build`)
- B) Que se copia desde el host, no desde otra imagen
- C) Que se copia desde Docker Hub, del repositorio `build`
- D) Que se renombra el fichero a `build`

Pregunta 54

Un Dockerfile multi-stage tiene una etapa `AS build` con herramientas de compilación de 800 MB, y una etapa final `FROM nginx:alpine` que solo copia los ficheros compilados. ¿Qué tamaño tendrá la imagen final publicada?

- A) 800 MB, porque incluye todo lo usado en el proceso
- B) El tamaño de la imagen final solo incluye lo que tiene esa última etapa (`nginx:alpine` + los ficheros copiados), no las herramientas de la etapa `build`
- C) La suma de ambas etapas
- D) Depende de cuántos `RUN` tenga la etapa `build`

Pregunta 55

¿Puede un Dockerfile multi-stage tener más de dos etapas (`FROM ... AS algo`)?

- A) No, el máximo son dos etapas
- B) Solo si todas usan la misma imagen base
- C) Sí, se pueden encadenar tantas etapas como haga falta, y cada una puede copiar de las anteriores
- D) No, Docker no soporta esa sintaxis

Pregunta 56

Si en un Dockerfile multi-stage olvidas poner `AS build` en la primera etapa, ¿puedes seguir referenciándola en `COPY --from=...` ?

- A) No, sin nombre no se puede referenciar nada de esa etapa
- B) Se usa automáticamente el nombre `default`
- C) Docker la ignora completamente si no tiene nombre
- D) Sí, se puede referenciar por su índice numérico (`--from=0` para la primera etapa, `--from=1` para la segunda...)

Pregunta 57

¿Qué ventaja tiene un build multi-stage frente a simplemente desinstalar las herramientas de compilación con `RUN` al final del mismo Dockerfile (de una sola etapa)?

- A) En una sola etapa, lo instalado y luego desinstalado sigue ocupando espacio en las capas intermedias; en multi-stage, esas capas ni siquiera forman parte de la imagen final
- B) Ninguna, el resultado final pesa exactamente igual en ambos casos
- C) El multi-stage es solo una cuestión de organización, no afecta al tamaño
- D) El multi-stage es más lento siempre

Bloque 8

Tags, versionado y latest

Pregunta 58

¿Qué es, técnicamente, el tag `latest` de una imagen en un registry como Docker Hub?

- A) Un indicador especial reservado por Docker que siempre apunta a la build más reciente y estable
- B) Un tag como cualquier otro, que por convención suele usarse para "la versión por defecto", pero que el autor de la imagen asigna manualmente igual que cualquier otro tag
- C) Una copia de seguridad automática de la imagen
- D) Un alias que apunta siempre a la primera versión publicada

Pregunta 59

Dos despliegues en distintos servidores hacen `docker pull mi-app:latest` en días distintos y obtienen comportamientos diferentes de la aplicación. ¿Cuál es la explicación más probable?

- A) Docker Hub está caído
- B) Es imposible, `latest` nunca cambia
- C) `latest` se volvió a publicar entre medias apuntando a una versión distinta de la imagen
- D) Los servidores tienen una versión de Docker distinta

Pregunta 60

¿Por qué en entornos de producción se prefiere desplegar con un tag de versión concreto (p. ej. `mi-app:2.3.1`) en vez de `mi-app:latest` ?

- A) Porque `latest` no se puede descargar en producción
- B) Porque las imágenes con `latest` ocupan más espacio en disco
- C) No hay ninguna diferencia real, es solo preferencia estética
- D) Porque con una versión fija sabes exactamente qué código está corriendo y evitas cambios inesperados al volver a desplegar

Pregunta 61

¿Qué formato de versionado es habitual en tags como `app:2.3.1` ?

- A) semver: mayor.menor.parche
- B) día.mes.año
- C) nombre.usuario.build
- D) No sigue ningún patrón estandarizado nunca

Pregunta 62

`docker tag mi-app:1.0 jose/mi-app:1.0` ¿crea una copia física nueva de la imagen en disco?

- A) Sí, duplica todos los datos de la imagen
- B) No, crea una nueva referencia (tag) que apunta al mismo ID de imagen ya existente; no se duplican datos
- C) Sí, pero solo la mitad de las capas
- D) Depende del tamaño de la imagen

Pregunta 63

Una imagen tiene dos tags, `mi-app:1.0` y `mi-app:stable`, apuntando ambos al mismo ID de imagen. Si haces `docker rmi mi-app:1.0`, ¿qué ocurre con `mi-app:stable`?

- A) También se elimina, porque comparten imagen
- B) Se renombra automáticamente a `1.0`
- C) Sigue existiendo: solo se elimina el tag `1.0`; la imagen en sí permanece mientras tenga al menos un tag o nombre que la referencie
- D) Da error porque no se puede borrar un tag compartido

Pregunta 64

¿Qué formato de nombre necesita una imagen para poder subirse a tu cuenta de Docker Hub con `docker push` ?

- A) Cualquier nombre, sin restricciones
- B) Debe llamarse obligatoriamente `latest`
- C) Debe tener extensión `.docker`
- D) Debe incluir el usuario o la organización de destino, como `tuusuario/nombre-imagen:tag`

Pregunta 65

Un alumno construye la imagen como `mi-app:1.0` pero quiere subirla como `jose/mi-app:1.0`. ¿Qué comando necesita ejecutar antes de `docker push jose/mi-app:1.0`?

- A) `docker tag mi-app:1.0 jose/mi-app:1.0`
- B) `docker build jose/mi-app:1.0`
- C) `docker rename mi-app:1.0 jose/mi-app:1.0`
- D) No hace falta nada más, `docker push mi-app:1.0` ya sirve

Pregunta 66

Si repites `docker build -t mi-app:1.0 .` tras cambiar el código, sin cambiar el tag `1.0`, ¿qué ocurre con el tag anterior?

- A) Se crean dos imágenes distintas con el mismo tag conviviendo
- B) El tag `1.0` pasa a apuntar a la imagen nueva; la anterior queda sin ese tag (puede quedar como "dangling" si no tenía otro tag)
- C) Docker lo rechaza, no se puede reutilizar un tag
- D) Se incrementa automáticamente a `1.1`

Bloque 9

Volúmenes y Bind Mount

Pregunta 67

¿Qué problema resuelve un volumen Docker?

- A) Acelerar la red del contenedor
- B) Reducir el consumo de CPU
- C) Persistir datos más allá del ciclo de vida de un contenedor concreto, y/o compartir datos entre host y contenedor
- D) Definir variables de entorno

Pregunta 68

Un contenedor de base de datos escribe sus ficheros en `/var/lib/mysql` sin ningún volumen asociado. Al hacer `docker rm` de ese contenedor, ¿qué pasa con esos datos?

- A) Se guardan automáticamente en la imagen original
- B) Se mueven al host automáticamente
- C) Se suben a Docker Hub
- D) Se pierden, porque desaparecen junto con el sistema de ficheros del contenedor

Pregunta 69

¿Qué diferencia principal hay entre un bind mount (`-v /ruta/host:/ruta/contenedor`) y un volumen gestionado (`docker volume create` + `-v nombre:/ruta`)?

- A) En el bind mount, tú eliges la ruta exacta del host; en el volumen gestionado, Docker administra internamente dónde se guardan los datos
- B) Son exactamente lo mismo con distinta sintaxis
- C) El bind mount no permite escritura desde el contenedor
- D) Los volúmenes gestionados no sirven para bases de datos

Pregunta 70

En el flujo de desarrollo con bind mount (`docker run -it -v $(pwd):/app -w /app mi-app bash`), si modificas un fichero de código en tu editor (fuera del contenedor), ¿necesitas reconstruir la imagen para ver el cambio dentro del contenedor?

- A) Sí, siempre hay que hacer `docker build` de nuevo
- B) No: el bind mount comparte la misma carpeta en ambas direcciones, así que el cambio se ve al instante dentro del contenedor
- C) Solo si cambias el `Dockerfile`
- D) Solo los viernes, cuando se reinicia la caché

Pregunta 71

¿Por qué, en el flujo con bind mount, la imagen "workspace" normalmente solo necesita reconstruirse cuando cambia el entorno (una dependencia del sistema, la versión del lenguaje...) y no cuando cambia el código de la aplicación?

- A) Porque Docker detecta automáticamente qué cambió y decide si reconstruir
- B) Porque el código nunca cambia en desarrollo
- C) Porque el código vive fuera de la imagen, montado por bind mount; cambiar el entorno sí exige reconstruir, pero el código no
- D) Porque las imágenes no pueden contener código nunca

Pregunta 72

Un equipo usa `docker volume create db_data` y monta ese volumen en el contenedor de su base de datos. Si eliminan el contenedor (sin `-v` en `docker rm`) y crean uno nuevo montando el mismo volumen, ¿qué pasa con los datos?

- A) Se pierden, porque el volumen estaba asociado solo al contenedor anterior
- B) Se duplican
- C) Hace falta restaurarlos manualmente desde una copia
- D) Siguen disponibles: el volumen gestionado existe independientemente del contenedor que lo monte

Pregunta 73

¿Qué comando lista los volúmenes Docker existentes en el sistema?

- A) `docker volume ls`
- B) `docker ps -v`
- C) `docker images --volumes`
- D) `docker inspect --volumes`

Pregunta 74

Si dos contenedores distintos montan el mismo volumen gestionado al mismo tiempo, ¿qué es cierto?

- A) No es posible, un volumen solo puede montarlo un contenedor
- B) Ambos pueden leer y escribir sobre los mismos datos compartidos por ese volumen
- C) Cada uno obtiene una copia independiente
- D) Solo el primero puede escribir, el segundo solo puede leer

Pregunta 75

¿Por qué normalmente no se usa un bind mount para los datos de una base de datos en producción, prefiriendo un volumen gestionado?

- A) Porque los bind mounts no permiten que una base de datos escriba
- B) Porque los bind mounts solo funcionan en Windows
- C) Porque el volumen gestionado lo administra Docker de forma más consistente entre entornos, sin depender de que exista una ruta concreta en cada host
- D) No hay ninguna razón real, es indistinto

Pregunta 76

Un alumno monta `-v $(pwd) : /app` , pero su Dockerfile también hace `COPY . /app` y `WORKDIR /app` . Al arrancar el contenedor con ese bind mount, ¿qué prevalece dentro de `/app` ?

- A) Lo que copió `COPY` durante el build, el bind mount no tiene efecto
- B) Se fusionan ambos contenidos automáticamente
- C) Docker da un error por conflicto
- D) El contenido del bind mount (la carpeta del host) sustituye/oculta lo que había en `/app` dentro de la imagen mientras el contenedor esté en marcha

Pregunta 77

¿Qué significa, en este bloque, que "la imagen contiene el entorno y el bind mount aporta el código"?

- A) Que el Dockerfile se centra en instalar lenguaje/dependencias del sistema, y el código real del proyecto se monta en tiempo de ejecución, sin necesitar estar dentro de la imagen
- B) Que el código también debe copiarse con `COPY` en el Dockerfile para que funcione
- C) Que no hace falta ningún Dockerfile si usas bind mount
- D) Que el bind mount sustituye por completo al concepto de imagen

Pregunta 78

¿Qué pasaría si, en vez de usar bind mount, copiaras el código con `COPY` dentro de la imagen y reconstruyeras la imagen cada vez que cambias una línea durante el desarrollo?

- A) Sería exactamente igual de rápido que con bind mount
- B) El ciclo de desarrollo sería mucho más lento, porque cada cambio pequeño obligaría a repetir `docker build`
- C) No sería posible hacerlo nunca
- D) El código no se vería reflejado nunca en el contenedor

Bloque 10

Redes y puertos

Pregunta 79

¿Qué hace `-p 8080:80` en `docker run` ?

- A) Limita el contenedor a 8080 MB de RAM
- B) Cambia el puerto interno del contenedor a 8080
- C) Publica el puerto 80 del contenedor en el puerto 8080 del host
- D) Bloquea el acceso a internet del contenedor

Pregunta 80

En `-p 8080:80`, ¿qué lado del mapeo corresponde al host?

- A) `80`, el segundo número
- B) Ambos corresponden al host
- C) Ninguno, ambos son del contenedor
- D) `8080`, el primer número

Pregunta 81

Por defecto, cuando se crea un contenedor sin indicar ninguna red concreta, ¿a qué red se conecta?

- A) A la red "bridge" por defecto que crea Docker
- B) A la red del host directamente (`--network host`)
- C) A ninguna, queda totalmente aislado de la red
- D) A la red de la nube del proveedor

Pregunta 82

Dos contenedores están conectados a una misma red Docker personalizada (`docker network create mi red`). Uno se llama `db` . ¿Cómo puede el otro contenedor conectarse a él sin usar IPs fijas?

- A) Usando `localhost`
- B) Usando el nombre del contenedor (`db`) como si fuera un hostname; Docker lo resuelve internamente en esa red
- C) No es posible sin publicar puertos al host
- D) Solo con la IP pública de internet

Pregunta 83

Si dos contenedores están en redes Docker distintas (sin ninguna en común), ¿pueden resolverse por nombre entre sí?

- A) Sí, siempre, Docker las conecta automáticamente
- B) Solo si ambos publican puertos al host
- C) No, para que se resuelvan por nombre necesitan compartir al menos una red
- D) Solo si tienen el mismo nombre de imagen

Pregunta 84

Intentas publicar dos contenedores distintos con `-p 3000:3000` cada uno, a la vez. ¿Qué ocurre con el segundo `docker run` ?

- A) Ambos arrancan sin problema, compartiendo el puerto
- B) El segundo contenedor cambia su puerto automáticamente al 3001
- C) El primer contenedor se detiene automáticamente para dejar paso al segundo
- D) Falla con un error de "puerto ya en uso" en el host

Pregunta 85

¿Qué opción publicaría automáticamente, con puertos aleatorios del host, todos los puertos que la imagen declaró con `EXPOSE` ?

- A) `-P` (mayúscula)
- B) `-p` sin argumentos
- C) `--publish-all=false`
- D) `--expose`

Pregunta 86

Un contenedor necesita comunicarse con otro que está en una red Docker distinta, y no se puede (ni se quiere) publicar puertos al host. ¿Cuál es la solución más directa?

- A) Reinstalar Docker
- B) Conectar ambos contenedores a una misma red Docker (p. ej. con `docker network connect`)
- C) Cambiar el Dockerfile del segundo contenedor
- D) Usar `docker build` de nuevo

Bloque 11

Docker Hub

Pregunta 87

¿Qué es Docker Hub?

- A) Un comando para crear contenedores
- B) Un tipo de volumen compartido entre contenedores
- C) Un registro (público, con opción privada de pago) donde se almacenan y distribuyen imágenes Docker
- D) El proceso daemon de Docker

Pregunta 88

¿Hace falta estar autenticado (`docker login`) para descargar una imagen pública de Docker Hub con `docker pull` ?

- A) Sí, siempre es obligatorio
- B) Solo los fines de semana
- C) Solo si la imagen pesa más de 1 GB
- D) No, las imágenes públicas pueden descargarse sin autenticarse

Pregunta 89

¿Qué son las "imágenes oficiales" de Docker Hub (como `nginx`, `python`, `node`)?

- A) Imágenes mantenidas/revisadas por Docker o por el propio proyecto, consideradas de mayor confianza
- B) Imágenes creadas automáticamente por cualquier usuario sin revisión
- C) Imágenes que ya no reciben actualizaciones nunca
- D) Imágenes exclusivas para cuentas de pago

Pregunta 90

¿Qué comando necesitas ejecutar antes de poder hacer `docker push` de una imagen por primera vez en una sesión?

- A) `docker build --push`
- B) `docker login`
- C) `docker pull --auth`
- D) No hace falta nada especial

Pregunta 91

Un desarrollador quiere mantener imágenes de su empresa fuera del acceso público, pero seguir usando Docker. ¿Qué opción tiene?

- A) No es posible, Docker Hub es siempre público
- B) Cifrar manualmente cada capa de la imagen
- C) Usar un repositorio privado (en Docker Hub de pago o un registry privado propio)
- D) Renombrar la imagen para que nadie la encuentre

Pregunta 92

Si haces `docker pull jose/mi-app` sin especificar tag, ¿qué versión descargas?

- A) Siempre falla sin tag explícito
- B) La primera versión publicada históricamente
- C) Todas las versiones a la vez
- D) La marcada como `latest` en ese repositorio de Docker Hub

Pregunta 93

Un compañero sube por error una imagen con una contraseña real incrustada en una variable `ENV`, a un repositorio público de Docker Hub. ¿Qué riesgo implica?

- A) Cualquiera que descargue la imagen puede inspeccionar su historial (p. ej. `docker history`) y ver esa información
- B) Ninguno, las variables `ENV` no son visibles desde fuera de un contenedor en marcha
- C) Docker Hub la oculta automáticamente al detectar texto sensible
- D) Solo es un problema si alguien ejecuta esa imagen

Pregunta 94

¿Qué pasos, en orden, describen correctamente publicar una imagen propia en Docker Hub (partiendo de una imagen ya construida localmente como `mi-app:1.0`)?

- A) `docker push mi-app:1.0` → `docker login` → `docker tag`
- B) `docker login` → `docker tag mi-app:1.0 usuario/mi-app:1.0` → `docker push usuario/mi-app:1.0`
- C) `docker pull` → `docker build` → `docker rm`
- D) `docker run` → `docker commit` → `docker stop`

Bloque 12

Organización de repos Git con Docker

Pregunta 95

En un flujo de trabajo con un "workspace" Docker reutilizable (p. ej. `laravel-workspace`) y un proyecto real montado dentro por bind mount, ¿cuántos repositorios git distintos suele haber, como mínimo?

- A) Ninguno, basta con un `.gitignore`
- B) Siempre uno solo, compartido
- C) Normalmente dos: uno para el workspace (entorno/Dockerfile) y otro para el proyecto real
- D) Tres obligatoriamente: workspace, proyecto y Docker Hub

Pregunta 96

¿Por qué el repositorio del "workspace" (con el `Dockerfile` del entorno) a menudo no necesita subirse a ningún sitio especial para compartirlo con otros?

- A) Porque Docker lo sube automáticamente a Docker Hub
- B) Porque no se puede subir un repositorio con un Dockerfile dentro
- C) Porque el workspace no puede tener historial de commits
- D) Porque muchas veces es solo una herramienta local de trabajo del desarrollador, mientras que lo que de verdad importa compartir es el código del proyecto

Pregunta 97

Si el proyecto real vive en `laravel-workspace/mi-proyecto`, y ambas carpetas tienen su propio `.git`, ¿el git de `laravel-workspace` llega a versionar los ficheros de `mi-proyecto` ?

- A) No: cada repositorio git gestiona su propia carpeta; para que `laravel-workspace` no intente versionar `mi-proyecto`, conviene excluirla (p. ej. en su `.gitignore`) o tratarla como un repositorio aparte
- B) Sí, automáticamente, porque está dentro
- C) Sí, pero solo si se hace `git add` explícitamente desde fuera
- D) Es imposible tener dos repositorios `.git` anidados

Pregunta 98

¿Qué ventaja principal ofrece mantener un único workspace reutilizable para varios proyectos del mismo framework, frente a un repositorio con Dockerfile propio por cada proyecto?

- A) Ninguna, siempre es peor
- B) Evita repetir la configuración del entorno (Dockerfile, dependencias del sistema) en cada proyecto nuevo del mismo stack
- C) Hace que cada proyecto pese menos en Docker Hub
- D) Es obligatorio por las buenas prácticas de Docker

Pregunta 99

¿En qué situación tiene más sentido que cada proyecto tenga su propio `Dockerfile` dentro de su propio repositorio, en vez de depender de un workspace externo?

- A) Nunca, siempre es mejor un workspace único
- B) Solo si el proyecto no usa Docker
- C) Cuando distintos proyectos necesitan entornos claramente distintos, o cuando se quiere que cualquiera pueda clonar el proyecto y tener el entorno listo sin pasos adicionales
- D) Solo cuando el proyecto no tiene base de datos

Pregunta 100

Resume el flujo típico de esta semana para desarrollar una aplicación propia en Docker, combinando Dockerfile y bind mount:

- A) `docker pull` → `docker rm` → `docker build`
- B) `docker run` → `docker commit` → `docker stop`
- C) `docker images` → `docker network create` → `docker volume create`
- D) Escribir el `Dockerfile` del entorno → `docker build -t workspace:1.0 .` →
`docker run -it -v <proyecto>:/app workspace:1.0 bash` → desarrollar montando el código

¡Suerte con el repaso!